

Comprehensive Guide: Two-Phase PageRank with Travel-Time Self-Loops

A Detailed Mathematical Breakdown

Contents

1	Introduction to the Problem	2
2	The Core Problem: Instantaneous Transitions	2
3	The Solution: Travel-Time Self-Loops (μ)	2
3.1	Mathematical Definition	2
3.2	The Phase Transition Matrix (P)	3
3.3	Example Calculation	3
4	The Two-Phase $2N \times 2N$ Block Matrix (M_{2N})	4
4.1	The Matrix Construction	4
5	The Teleportation Vector (E_{2N})	5
5.1	Constructing E_N	5
5.2	Embedding into $2N$ Space	5
6	The Power Iteration Algorithm	5
7	Extracting the Final Distribution (v_{final})	6
8	Conclusion	6

1 Introduction to the Problem

The goal of this project is to predict macroscopic traffic congestion across a massive urban road network (e.g., Salt Lake City with 99,716 distinct road links) based on a sparse sample of GPS trajectories.

The process involves two main steps:

1. **Smoothing:** Raw, sparse trajectory counts are spatially smoothed using graph diffusion to create a continuous probability field called the *ground-truth popularity distribution* (E_N).
2. **PageRank Simulation:** A customized PageRank algorithm runs a random walk on the road network graph. It uses E_N as its teleportation vector to generate a stationary distribution v , which represents the predicted traffic volume for every road segment.

While standard PageRank models network flow well in the aggregate, it drastically underpredicts traffic on the busiest, longest roads (the “Top-100” error). This document explains the exact mathematical mechanics of the **Two-Phase PageRank with Travel-Time Self-Loops**, an algorithm explicitly designed to solve this physical discrepancy.

2 The Core Problem: Instantaneous Transitions

Standard PageRank is a discrete-time Markov chain. In every iteration step, all probability mass (simulated vehicles) at a node must either move to its successors or teleport.

In a road network, this means a vehicle transitions from a 1 km highway segment to the next link in the exact same mathematical “time step” as a vehicle transitioning through a 10 meter intersection.

Because standard PageRank treats every transition as instantaneous, it ignores **dwelt time**. Traffic physically accumulates on long, slow roads because vehicles spend more time there. By forcing mass to flow identically regardless of road length, standard PageRank causes excessive diffusion, washing out the concentration peaks on major arterial roads and highways.

3 The Solution: Travel-Time Self-Loops (μ)

To fix this, we introduce a physical constraint: **Travel-Time Self-Loops**.

Instead of forcing all mass to exit a link in one step, we add a self-loop to every single link i . This self-loop dictates what fraction of traffic *stays* on the link during an iteration. The strength of this self-loop is directly proportional to the physical time it takes to traverse the link.

3.1 Mathematical Definition

For every link i , we define its traversal time as $t_i = \frac{\text{length}_i}{\text{speed}_i}$. We introduce a global scaling parameter, μ (mu), which controls the overall strength of dwell time in the model.

The raw self-loop weight for link i is calculated as:

$$\text{self_weight}_i = \mu \cdot \frac{\text{length}_i}{\text{speed}_i} \quad (1)$$

Let w_{ij} be the raw weight of moving from link i to a successor link j . By default, when edge attribute tuning parameters like α_s and α_l are set to 0.0, $w_{ij} = 1.0$ for all valid outgoing edges. The total weight leaving link i is therefore the sum of the self-loop weight and all outgoing edge weights:

$$\text{Total_Weight}_i = \text{self_weight}_i + \sum_{j \in \text{successors}(i)} w_{ij} \quad (2)$$

3.2 The Phase Transition Matrix (P)

The base $N \times N$ transition matrix P governs flow through the network. The probability of transitioning from link i to link j (where j can be i itself) is strictly normalized by the total weight.

The **self-loop probability** (the probability of staying on link i):

$$P_{i,i} = \frac{\text{self_weight}_i}{\text{Total_Weight}_i} \quad (3)$$

The **forward probability** (the probability of moving to successor j):

$$P_{i,j} = \frac{w_{ij}}{\text{Total_Weight}_i} \quad (4)$$

3.3 Example Calculation

Consider two roads in a network where $\mu = 20$:

1. A Highway Segment: (Length = 500m, Speed = 25m/s, 3 successors)

- Traversal time = $500/25 = 20$ seconds.
- Self-weight = $\mu \times 20 = 20 \times 20 = 400$.
- Successor weights = $1 + 1 + 1 = 3$.
- Total weight = $400 + 3 = 403$.
- **Self-loop probability** ($P_{i,i}$) = $400/403 \approx 99.25\%$.
- Forward probability per successor = $1/403 \approx 0.25\%$.

Result: Because the highway is long, 99.25% of its mass stays on it per iteration.

2. A Short Connector: (Length = 50m, Speed = 10m/s, 3 successors)

- Traversal time = $50/10 = 5$ seconds.

- Self-weight = $\mu \times 5 = 20 \times 5 = 100$.
- Successor weights = 3.
- Total weight = 103.
- **Self-loop probability** ($P_{i,i}$) = $100/103 \approx 97.08\%$.

Result: The connector retains less mass than the highway. Over 100 iterations, the highway retains $(0.9925)^{100} \approx 47\%$ of its initial mass, while the connector retains only $(0.9708)^{100} \approx 5\%$. **This perfectly models spatial traffic accumulation.**

4 The Two-Phase $2N \times 2N$ Block Matrix (M_{2N})

Traffic is directional, but vehicles eventually stop or park, diffusing into local neighborhood roads. To model this, the algorithm splits the network into two identical “phases” or layers, doubling the matrix size from N to $2N$.

1. **Up Phase (Forward Flow):** Represents vehicles actively driving. They follow the directed road network.
2. **Down Phase (Local Dispersal):** Represents vehicles slowing down, parking, or entering local areas.

While the physical network transitions (P) are technically identical in both phases (because $\alpha_s = \alpha_l = 0.0$), the way mass flows *between* the phases is governed by the matrix block structure.

4.1 The Matrix Construction

We construct a $2N \times 2N$ block transition matrix M_{2N} . A transition rate parameter, β (beta), determines how quickly mass “falls” from the Up phase to the Down phase.

$$M_{2N} = \begin{bmatrix} (1 - \beta) \cdot P & \beta \cdot I \\ 0 & P \end{bmatrix} \quad (5)$$

Let’s break down these four $N \times N$ quadrants:

- **Top-Left** ($(1 - \beta) \cdot P$): If a vehicle is in the Up phase, it has a $(1 - \beta)$ probability of staying in the Up phase and moving to a successor (or self-looping) according to P .
- **Top-Right** ($\beta \cdot I$): If a vehicle is in the Up phase, it has a probability β of falling into the exact same link i in the Down phase (I is the Identity matrix). This is a vertical transition between layers.
- **Bottom-Left** (0): Vehicles in the Down phase can never return to the Up phase.
- **Bottom-Right** (P): Once in the Down phase, vehicles continue to move (or self-loop) according to P indefinitely.

With $\beta = 0.90$, mass aggressively transitions from the Up phase to the Down phase after roughly one hop, preventing excessive forward diffusion while the μ self-loops handle the physical dwell time.

5 The Teleportation Vector (E_{2N})

In standard PageRank, walkers occasionally “teleport” to a random node to ensure the Markov chain is strongly connected. In this project, the teleportation vector is not random; it is heavily biased toward the empirical ground truth.

5.1 Constructing E_N

E_N is an N -dimensional probability vector derived from the smoothed trajectory counts ($\gamma = 0.26$). It represents the empirically observed distribution of traffic. $\sum_{i=1}^N E_N[i] = 1$.

Note on Legacy Parameters: In older versions of the algorithm, researchers tried artificially squaring or boosting the values in E_N for the top 100 links (using parameters ‘tau’ and ‘top_k.boost’). Because the μ self-loop physically solves the problem, these parameters are now disabled (‘tau=1.0’, ‘top_k.boost=1.0’), meaning E_N represents the unadulterated smoothed data.

5.2 Embedding into 2N Space

Because our transition matrix is $2N \times 2N$, our teleportation vector must also be $2N$. We want all teleporting vehicles to “spawn” actively driving. Therefore, all teleportation mass is injected strictly into the Up phase.

$$E_{2N} = \begin{bmatrix} E_N \\ \mathbf{0}_N \end{bmatrix} \quad (6)$$

Where $\mathbf{0}_N$ is a vector of N zeros.

6 The Power Iteration Algorithm

To find the predicted traffic distribution, we simulate the Markov chain until it reaches a stationary state using Power Iteration.

Let $v_{2N}^{(k)}$ be the state vector at iteration k . The PageRank update rule is:

$$v_{2N}^{(k+1)} = d \cdot M_{2N}^T \cdot v_{2N}^{(k)} + (1 - d) \cdot E_{2N} \quad (7)$$

Where:

- $d = 0.80$ is the **damping factor**. At each step, 80% of the mass moves according to the transition matrix M_{2N} , and 20% of the mass teleports according to E_{2N} .
- M_{2N}^T is the transpose of the transition matrix (since we represent probability distributions as column vectors).

The iteration continues until the L1 norm difference between consecutive states is microscopically small ($\|v^{(k+1)} - v^{(k)}\|_1 < 10^{-6}$).

7 Extracting the Final Distribution (v_{final})

Once the power iteration converges, we have a $2N$ -dimensional stationary vector v_{2N} . However, our physical road network only has N links.

To get the final macroscopic predicted traffic for link i , we simply sum the probability mass across both the Up and Down phases for that link:

$$v_{final}[i] = v_{up}[i] + v_{down}[i] \quad (8)$$

$$v_{final} = \frac{v_{final}}{\sum v_{final}} \quad (\text{Renormalization}) \quad (9)$$

8 Conclusion

By injecting physical reality into a mathematical algorithm, the Two-Phase PageRank with Travel-Time Self-Loops elegantly solves the "Top-100 Error."

Rather than arbitrarily forcing traffic onto large roads, the μ parameter transforms the transition matrix into a continuous-time approximation. It forces the simulation to acknowledge that a 1km highway physically retains a vehicle 100 times longer than a 10m intersection, allowing realistic traffic concentrations to emerge naturally through the Power Iteration process.